

Формальная верификация инвариантов TicketLock в Verus и конфигурация динамического покрытия кода для ОС на базе Rust

Введение в формальную верификацию высоконадежных операционных систем

Разработка современных операционных систем все чаще смещается в сторону методологий, обеспечивающих безопасность памяти и корректность параллельного выполнения на уровне архитектуры и компиляции. Язык программирования Rust стал одним из ведущих инструментов для этих целей, предлагая мощную систему типов, которая проверяет линейность (linearity) и заимствование (borrowing), что позволяет коду безопасно управлять памятью без использования сборщика мусора. Это делает Rust исключительно подходящим для разработки низкоуровневых высоконадежных систем, таких как ядро операционной системы `piOS`. Однако, несмотря на то что компилятор Rust предотвращает широкий класс уязвимостей, связанных с безопасностью памяти и гонками данных (data races), он не может статически верифицировать более широкие свойства функциональной корректности или сложные, неограниченные параллельные алгоритмы, такие как примитивы честной блокировки (fair locks).

Для устранения этого пробела в конвейер разработки интегрируются инструменты дедуктивной формальной верификации. Verus — это передовой инструмент на базе SMT-решателя (в частности, Z3), разработанный специально для формальной верификации программ на языке Rust. Позволяя программистам выражать доказательства и спецификации непосредственно с использованием синтаксиса Rust, Verus использует линейные типы и проверку заимствований Rust для управления линейно типизированными разрешениями (permissions). Это гарантирует, что исполняемый код строго соответствует математическим спецификациям во всех возможных состояниях выполнения.

При верификации сложных параллельных ресурсов, таких как TicketLock (задача [Verus Fix] WP-051 в репозитории 3-14AI/piOS), разработчики должны строго определять конечные автоматы (state machines), инварианты и теньевые состояния (ghost states), чтобы доказать такие свойства, как честность (fairness), взаимное исключение и отсутствие состояний гонки. В частности, анализ `kernel/src/sync.rs` требует глубокого понимания того, как SMT-решатель обрабатывает обязательства для свойств `waiting_implies_storage` и условий безопасности параметров хранения (`deposit` и `withdraw`). Одновременно с этим формальная верификация должна дополняться эмпирическим динамическим тестированием, чтобы гарантировать, что пограничные случаи в неверифицированных слоях, таких как среды выполнения WebAssembly (например, `kernel/src/wasm/runtime.rs`), работают должным образом. Инструменты покрытия кода, в частности Cargo Tarpaulin, имеют решающее значение для выявления

непротестированных ветвей и невыполненных путей инструкций. Оркестровка этих двух парадигм — статических формальных доказательств через Verus и динамического отслеживания покрытия через Cargo Tarpaulin — формирует основу инженерии высоконадежных систем.

В данном исследовании проводится исчерпывающий теоретический и практический анализ внедрения инвариантов Ticket Lock в рамках архитектуры Verus. Кроме того, предоставляется комплексный диагностический анализ аномалий среды и набора инструментов, связанных с нераспознаванием команды cargo-tarpaulin в средах no_std, что обеспечит максимальное тестовое покрытие для ядра piOS.

Архитектура Verus: Режимы, Ghost State и линейные типы

Для разрешения проблем с математическими доказательствами TicketLock, необходимо детально рассмотреть внутреннюю архитектуру инструмента Verus. Verus вводит парадигму, в которой спецификации, доказательства и исполняемый код сосуществуют в одном синтаксисе Rust, разделенные инновационной системой режимов (mode system). Система режимов Verus строго разделяет логику на три категории:

1. **Режим spec (Спецификации):** Используется для чистых математических спецификаций, которые не проверяются на линейность и заимствование. Код в этом режиме не транслируется в исполняемые инструкции машинного кода. Он служит исключительно абстрактной моделью поведения системы.
2. **Режим proof (Доказательства):** Содержит дедуктивную логику, необходимую для выполнения обязательств SMT-решателя, направляя базовую систему автоматического доказательства теорем Z3 через сложные индукции. Этот режим проверяется на линейность, что позволяет доказательствам манипулировать логическими ресурсами, но он также стирается на этапе компиляции.
3. **Режим exec (Исполняемый код):** Содержит фактический исполняемый код Rust, который компилируется в бинарный файл. Этот код строго проверяется как компилятором Rust, так и SMT-решателем Verus на соответствие условиям, заданным в spec и proof.

Формализуя линейность, заимствование и режимы в небольшом лямбда-исчислении, Verus доказывает типобезопасность и завершаемость как спецификаций, так и доказательств.

Сепарационная логика и теневые объекты (Ghost Objects)

Для рассуждений о предложениях сепарационной логики (separation logic) и параллелизме с разделяемой памятью Verus использует примитивные теневые объекты (ghost objects). К ним относятся разрешения памяти (memory permissions), теневое состояние на основе алгебры ресурсов (RA-based ghost state) и открываемые инварианты (openable invariants). Теневое состояние является чисто логическим; оно существует только во время верификации во время компиляции для отслеживания разрешений и состояния системы, но полностью стирается компилятором, не создавая никаких накладных расходов во время выполнения.

При верификации параллельных структур данных, таких как блокировки, проблема заключается в доказательстве того, что несколько потоков координируют свой доступ без нарушения правил владения. В Rust стандартные абстракции владения не могут выразить сложные протоколы координации, где состояние блокировки распределено между

несколькими атомарными переменными. Verus решает эту проблему с помощью макроса `tokenized_state_machine!`.

Токенизированный конечный автомат моделирует параллельный протокол как математический автомат. Переходы (transitions) между состояниями в этой машине генерируют или потребляют «токены», которые представляют собой линейные логические разрешения. Поток должен обладать правильным токеном для выполнения определенного действия (например, для изменения счетчика или доступа к защищенным данным), что математически гарантирует, что никакие два потока не могут одновременно обладать конфликтующими разрешениями.

Теоретический анализ алгоритма Ticket Lock

Чтобы понять обязательства SMT-решателя, невыполнение которых приводит к ошибке WP-051, необходимо явно определить операционную семантику Ticket Lock и сравнить ее с более простыми примитивами синхронизации.

Стандартный Spinlock (циклическая блокировка) работает за счет использования одной атомарной булевой переменной или целочисленного значения. Потoki многократно выполняют инструкцию сравнения с обменом (CAS — compare-and-swap), пока успешно не переведут блокировку из свободного состояния в удерживаемое. Несмотря на простоту реализации, стандартные циклические блокировки лишены свойства честности (fairness). В условиях высокой конкуренции (high contention) один поток может захватывать блокировку много раз подряд, в то время как другие потоки будут страдать от голодания (starvation), что приводит к непредсказуемым задержкам и сценариям инверсии приоритетов.

Алгоритм Ticket Lock решает эту проблему, обеспечивая строгую честность по принципу «первым пришел — первым обслужен» (FIFO). Архитектура имитирует физическую очередь, используя два атомарных целочисленных счетчика: `next` (выдача билетов) и `serving` (текущий обслуживаемый билет).

Механизм работы Ticket Lock состоит из следующих этапов:

1. **Захват (Acquire):** Когда поток пытается захватить блокировку, он выполняет атомарную инструкцию `fetch_add` на счетчике `next`, получая уникальный, строго монотонно возрастающий номер билета.
2. **Ожидание (Spinning):** Затем поток входит в цикл (spin), непрерывно считывая значение счетчика `serving` с семантикой памяти Acquire. Цикл прерывается только тогда, когда значение `serving` совпадает с полученным номером билета потока.
3. **Освобождение (Release):** При освобождении блокировки поток просто увеличивает счетчик `serving` на единицу с семантикой памяти Release, плавно передавая право собственности следующему потоку в очереди без необходимости дополнительных атомарных операций сравнения и обмена.

Характеристика	Стандартный Spinlock	Ticket Lock
Атомарные операции	Compare-and-Swap (CAS) при каждой попытке	Одиночный Fetch-and-Add, далее обычные атомарные чтения
Гарантия честности	Отсутствует (подверженность голоданию потоков)	Строгое упорядочивание FIFO
Накладные расходы при конкуренции	Высокие (инвалидация кэш-линий при каждом CAS)	Умеренные (все потоки читают одну переменную)

Характеристика	Стандартный Spinlock	Ticket Lock
		serving)
Переменные состояния	Одиночный AtomicBool	Два независимых AtomicUsize (next и serving)

Хотя Ticket Lock решает проблему голодания, формальная верификация его корректности в рамках `tokenized_state_machine!` крайне сложна. SMT-решатель Z3 должен установить индуктивный инвариант, который математически связывает значения счетчиков `next` и `serving` с физическим владением защищаемыми данными (что выражается в Verus через протоколы хранения).

Формализация инвариантов TicketLock в Verus (WP-051)

Конкретная ошибка, с которой столкнулся разработчик piOS в файле `kernel/src/sync.rs`, связана с невозможностью доказать инвариант `waiting_implies_storage` и условия безопасности для параметров `deposit` и `withdraw`. В токенизированном конечном автомате Verus переходы, содержащие операторы `withdraw` (изъятие) или `deposit` (внесение), кодируются в недетерминированные функции обмена внутри разделяемого состояния. Чтобы удовлетворить условиям безопасности, SMT-решатель требует явных индуктивных инвариантов, которые остаются истинными при всех возможных чередованиях выполнения потоков. Инвариант является индуктивным, если он выполняется при инициализации конечного автомата и математически сохраняется при каждом допустимом переходе состояния (state transition). Если инвариант не является строго индуктивным, решатель Z3 не сможет снять доказательства, что приведет к ошибкам компиляции, наблюдаемым разработчиком.

Математика инварианта `waiting_implies_storage`

Фундаментальная концепция `waiting_implies_storage` (ожидание подразумевает наличие хранилища) диктует следующее: если блокировка в настоящее время не принадлежит ни одному потоку, либо если потоки ожидают своей очереди, SMT-решатель должен быть в состоянии безоговорочно доказать, что теневого токена (ghost token), представляющий защищенный ресурс, физически находится во внутреннем пуле хранилища автомата блокировки.

Для Ticket Lock незанятое состояние абстрактно определяется как условие, при котором счетчик `next` равен счетчику `serving`, при условии, что не было выдано никаких билетов, которые в настоящее время ожидают. Однако это определение недостаточно для конкурентной системы, поскольку сразу несколько потоков могут держать билеты, значения которых больше или равны `serving`. Следовательно, инвариант должен быть определен как комплексная связь между множеством выданных билетов, счетчиком `serving` и местоположением теневого токена.

Конечный автомат должен отслеживать логическое мультимножество (или отображение) всех выданных билетов. Обозначим $T_{\{next\}}$ как значение счетчика `next`, а $T_{\{serving\}}$ — как значение счетчика `serving`.

Индуктивные инварианты, необходимые для Verus, должны явно утверждать следующие логические предложения:

1. **Ограничение монотонности:** Счетчик `next` всегда должен быть больше или равен счетчику `serving`. Формально: $\forall \text{forall } \text{states}, T_{\text{next}} \geq T_{\text{serving}}$. Это базовое условие предотвращает переполнение, при котором `serving` логически опережает выданные билеты.
2. **Связь билетов и потоков:** Существует взаимно однозначное соответствие между математическим диапазоном $\$$ Проблема заключается в том, что SMT-решатель не может автоматически сделать вывод о том, что токен *доступен* для изъятия, если только инвариант не гарантирует это.

Следовательно, для разрешения ошибки разработчик должен сформулировать инвариант, утверждающий, что если $T_{\text{next}} > T_{\text{serving}}$ и поток, удерживающий билет T_{serving} , еще не завершил свой переход `acquire` (т.е. его локальный токен состояния указывает на этап ожидания), ресурсный токен T по-прежнему *остается* в хранилище блокировки. Только после того, как переход `acquire` срабатывает, токен перемещается в локальный контекст выполнения потока, и инвариант автомата отражает, что хранилище пусто. Для успешной верификации в Verus необходимо использовать логические переменные (например, `Option<T>` в состоянии машины), чтобы указать присутствие ресурса:

- Если хранилище равно `Some(data)`, это означает, что ни один поток в данный момент не выполняет критическую секцию.
- Если хранилище равно `None`, это означает, что поток с билетом $t = T_{\text{serving}}$ успешно выполнил переход `acquire` и удерживает данные.

Разрешение условий безопасности для параметров `Deposit` и `Withdraw`

Условия безопасности для параметров хранения (`storage parameters`) неразрывно связаны с шаблонами защиты (`guard patterns`), используемыми в конечном автомате. Оператор, использующий шаблон защиты, гарантирует, что шаги в абстрактной модели могут выполняться исключительно тем потоком, которому принадлежит соответствующий токен защиты.

Чтобы исправить ошибку WP-051 в `piOS`, определения переходов должны быть структурно переработаны. Переход `release` (освобождение) должен быть определен таким образом, чтобы он атомарно связывал инкремент счетчика `serving` с внесением (`deposit`) теневого токена.

В формальных моделях, если конечный автомат отделяет внесение токена от продвижения счетчика T_{serving} , SMT-решатель Z3 обнаружит промежуточное логическое состояние, в котором блокировка технически свободна (следующий поток видит, что его билет совпадает с `serving`), но токен ресурса еще не был помещен обратно в пул хранилища. Это создает логическое состояние гонки (`logical race condition`) в модели доказательства, что приводит к сбою условий безопасности параметров хранения.

Правильная формулировка в Verus требует **единого атомарного перехода** для `release`. Этот переход должен:

1. Требовать (через `require`), чтобы поток отказался от своего токена эксклюзивного владения.
2. Выполнить `deposit` защищенных данных обратно во внутренний пул хранилища конечного автомата.
3. Одновременно выполнить обновление состояния (команду `update`), которое увеличивает T_{serving} на единицу.

Объединив эти действия в одно атомарное предложение SMT, обязательства доказывающего устройства (prover) для параметров депозита удовлетворяются. Поток не может увеличить `serving`, не предоставив доказательство владения защищенными данными (через передачу токена), что математически исключает гонки данных.

Стратегии инстанцирования кванторов в Z3

Важной технической деталью, о которой следует помнить при работе с Verus в `kernel/src/sync.rs`, является то, что Z3 в значительной степени полагается на инстанцирование кванторов для решения доказательств, включающих диапазоны целых чисел (например, очередь билетов). Если в инвариантах `TicketLock` используются универсальные кванторы (например, `forall [t: nat]...`), решатель может столкнуться с тайм-аутами (timeouts) или неспособностью инициировать правильные триггеры (triggers). Разработчик должен убедиться, что инвариант `waiting_implies_storage` написан с использованием точных экзистенциальных и универсальных кванторов, сопровождаемых явными аннотациями `#[trigger]` в Verus. Это заставляет решатель проверять условие хранения специально для границы билета `t = T_{serving}`, а не пытаться доказать состояние хранения для всего неограниченного пространства целых чисел.

Динамический анализ: Стратегии покрытия кода для riOS

В то время как формальная верификация с использованием Verus обеспечивает математическую уверенность в структурной логике примитивов, таких как `TicketLock`, операционные системы полагаются на многочисленные уровни взаимодействия с оборудованием, внешние библиотеки и среды выполнения, которые невозможно полностью смоделировать в SMT-решателе из-за проблемы взрыва состояний (state explosion). Среда выполнения `WebAssembly` (`kernel/src/wasm/runtime.rs`), упомянутая в запросе, является классическим примером сложного компонента, где дедуктивная верификация должна быть дополнена исчерпывающим динамическим тестированием. Стремление добиться максимального покрытия тестами в `kernel/src/lib.rs` и `Wasm`-рантайме требует надежного инструментария. `Cargo Tarpaulin` является стандартным инструментом отчета о покрытии кода для системы сборки `Cargo`. Он инструментирует скомпилированный бинарный файл для отслеживания покрытия строк (line coverage), покрытия ветвей (branch coverage) и состояний выполнения. Однако интеграция инструментов покрытия в среду ядра (kernel-level) и `WebAssembly` представляет собой ряд специфических архитектурных проблем.

Диагностика аномалии интеграции Cargo Tarpaulin ("Не распознано cargo")

Заявленная проблема — когда команда `cargo-tarpaulin` не распознается менеджером пакетов `cargo`, несмотря на то, что она была установлена ранее, — является фундаментальным нарушением в конвейере непрерывной интеграции (CI) или локальной среде разработки.

Когда вызывается команда, такая как `cargo tarpaulin`, менеджер пакетов `Cargo` не

выполняет встроенную подпрограмму; вместо этого он делегирует команду, выполняя поиск в системном окружении хоста исполняемого бинарного файла с явным именем cargo-tarpaulin. Cargo полностью полагается на системную переменную среды \$PATH для поиска этих сторонних подкоманд (subcommands).

Неспособность Cargo распознать установку Tarpaulin почти всегда проистекает из двух категорий проблем: неправильной конфигурации путей среды или тихих сбоев компиляции на этапе установки.

1. Проблема расширения \$PATH и CI сред: При выполнении команды cargo install cargo-tarpaulin бинарный файл компилируется и помещается в каталог бинарных файлов Cargo, который обычно находится по пути \$HOME/.cargo/bin в UNIX-подобных системах (Linux, macOS). Если этот конкретный каталог отсутствует в переменной среды \$PATH оболочки, операционная система физически не может найти исполняемый файл, побуждая Cargo возвращать ошибку «нет такой команды» (no such command).

В автоматизированных средах непрерывной интеграции (таких как GitHub Actions, GitLab CI или CircleCI) эта проблема усугубляется механизмами расширения оболочки (shell expansion). Скрипты CI часто пытаются динамически добавлять пути, используя символ тильды (например, echo "~/.cargo/bin" >> \$GITHUB_PATH). Однако инъекторы путей среды CI часто не расширяют тильду в абсолютный путь к домашнему каталогу, в результате чего в ограничения пути добавляется буквальная нефункциональная строка ~/.cargo/bin.

Решение: Требуется использование абсолютных путей, таких как /home/runner/.cargo/bin, или использование системных переменных среды, таких как \$HOME (например, echo "\$HOME/.cargo/bin" >> \$GITHUB_PATH), чтобы гарантировать правильное расширение пути оболочкой.

2. Тихие сбои компиляции и системные зависимости: Вторая причина отсутствия распознавания команды — прерванный процесс установки, который разработчик по ошибке считает успешным. Tarpaulin опирается на системные библиотеки C для выполнения трассировки процессов и анализа памяти операционной системы. В частности, компиляция Tarpaulin из исходного кода часто зависит от заголовков разработки OpenSSL (например, пакета libssl-dev в Ubuntu/Debian или openssl-devel в Fedora/RHEL).

Если эти базовые библиотеки C отсутствуют в хост-системе, процесс cargo install прервется, обычно выдавая ошибку компиляции, связанную с пакетом rust-openssl. Если разработчик пропускает этот вывод ошибки в журнале терминала (или если CI кэширует неудачный слой), он попытается запустить cargo tarpaulin, предполагая, что бинарный файл существует, что вызовет вышеупомянутое исключение отсутствия команды. Разработчику iOS необходимо очистить кэш Cargo (cargo cache -a) и повторить установку, внимательно следя за процессом линковки OpenSSL.

Специфика сбора покрытия в средах no_std и Wasm

Максимизация покрытия в kernel/src/wasm/runtime.rs и базовом модуле kernel/src/lib.rs вносит сложности, уникальные для системного программирования. Ядра операционных систем компилируются с атрибутом #![no_std], что означает, что они не связываются со стандартной библиотекой Rust (std), которая опирается на API базовой операционной системы для выделения памяти, работы с потоками и системных вызовов.

По умолчанию Cargo Tarpaulin работает с использованием механизма трассировки Ptrace в Linux. Этот бэкенд полагается на сигналы UNIX (в частности, SIGTRAP) для перехвата моментов, когда достигаются точки инструментирования (breakpoints). Внедрение точек

останова в исполняемые файлы и трассировка их выполнения требуют инструкций, специфичных для процессора и хостовой ОС. Поскольку бинарный файл ядра `no_std` (piOS) предназначен для работы на «голом железе» (bare metal) или в гипервизоре без базовой ОС Linux для обработки системных вызовов `ptrace`, запуск Tarpaulin непосредственно на необработанном бинарном файле ядра структурно несовместим. Для достижения максимального тестового покрытия в piOS разработчик должен стратегически изолировать тестируемую логику. Это достигается путем написания модульных тестов, которые выполняются в хост-среде (host-hosted environment), а не в среде bare-metal. Использование директив `#[cfg(test)]` позволяет условно подключать стандартную библиотеку исключительно во время тестовых запусков, что позволяет компилировать логические функции ядра (такие как конечные автоматы времени выполнения `Wasm` и алгоритмы примитивов синхронизации, включая не зависящие от аппаратуры части `TicketLock`) в виде стандартных бинарных файлов Linux. Только после этого Tarpaulin сможет успешно подключиться через `ptrace` и отслеживать выполнение.

Механизм Tarpaulin	Принцип инструментирования	Применение и ограничения в ОС
Ptrace (По умолчанию)	Сигналы SIGTRAP ОС через аппаратные точки останова	Лучше всего подходит для стандартных тестов Linux. Не работает при выполнении тестов на bare-metal или при активной манипуляции прерываниями.
LLVM Coverage (--engine llvm)	Профилирующие счетчики, внедренные компилятором	Высокая точность. Лучше справляется со сложными макросами. Не сохраняет данные , если тест завершается с ненулевым кодом (например, тесты <code>should_panic</code>).

Преимущества LLVM Coverage для макросов Verus

Для очень сложных макросов, таких как `tokenized_state_machine!` (используемых Verus в `kernel/src/sync.rs`), бэкэнд `ptrace` по умолчанию может испытывать трудности с точным сопоставлением бинарных инструкций с исходными строками исходного кода Rust. В этих сценариях переключение бэкэнда Tarpaulin на инструментирование покрытия LLVM с использованием флага `--engine llvm` обеспечивает значительно более высокую точность. Движок LLVM полагается на инструкции профилирования, внедренные непосредственно бэкэндом компилятора Rust (`rustc`), гарантируя, что расширения макросов (`macro expansions`) отслеживаются логически, а не полагаются на корреляцию строк отладочных символов (DWARF). Однако разработчики должны учитывать важный нюанс: бэкэнд инструментовки LLVM полностью сбрасывает данные о покрытии, если тестовый процесс завершается с ненулевым кодом возврата. Это означает, что тесты, использующие атрибут `#[should_panic]` (которые часто применяются для проверки правильности обработки ошибочных состояний в `Wasm`-рантайме), должны быть изолированы в отдельный тестовый прогон или обрабатываться с особой осторожностью при анализе метрик покрытия, иначе покрытие для этих файлов упадет до нуля.

Влияние уровней оптимизации компилятора

Кроме того, флаги оптимизации компилятора сильно искажают отчеты о покрытии. Для работы системы покрытия кода необходимо, чтобы компилятор генерировал различные инструкции ЦП для *каждой* строки исходного кода. Если проект компилируется с включенными слоями оптимизации (например, через настройку `RUSTFLAGS="-C opt-level=2"` или в рамках профиля `release`), компилятор будет выполнять встраивание функций (inlining), разворачивание циклов (loop unrolling) и устранение избыточных выборок переменных (dead code elimination).

В результате строки кода, которые логически выполняются, не будут иметь соответствующих инструкций ассемблера в финальном бинарном файле, что приведет к тому, что Tarpaulin некорректно пометит эти строки как «непокрытые». Чтобы обеспечить максимальную видимость покрытия для `kernel/src/lib.rs` и среды выполнения Wasm, разработчик `piOS` должен убедиться, что профиль сборки по умолчанию использует нулевую оптимизацию (`opt-level=0`) во время работы конвейера анализа покрытия Tarpaulin.

Заключение и рекомендации

Успешная разработка высоконадежной операционной системы `piOS` на языке Rust опирается на овладение как математической строгостью дедуктивной верификации, так и логистической точностью конвейеров динамического тестирования.

Для разрешения проблемы [Verus Fix] WP-051, связанной с примитивом `TicketLock`, формулировка токенизированного конечного автомата должна быть структурно усовершенствована:

1. SMT-решатель Z3 требует явных индуктивных инвариантов, которые логически связывают физические атомарные счетчики (`next` и `serving`) с логическим пулом хранения теневого состояния (`ghost state`).
2. Инвариант `waiting_implies_storage` доказывается путем явного ограничения счетчиков (`T_{next}` и `T_{serving}`) и строгой дизъюнктивной спецификации: теневого токена защищенных данных остается в пуле хранилища автомата *до тех пор*, пока поток, чей билет совпадает с `serving`, не выполнит переход изъятия (`withdraw`). Использование макросов `#[trigger]` обязательно для управления кванторами.
3. Условия безопасности для параметров `deposit` и `withdraw` удовлетворяются только в том случае, если переход освобождения блокировки (`release`) *атомарно* объединяет внесение токена в хранилище с продвижением счетчика `serving`. Разделение этих действий создает логическую гонку данных, которую SMT-решатель справедливо отклоняет.

В контексте максимизации тестового покрытия Wasm-рантайма и базовых библиотек ядра:

1. Устранение ошибки нераспознавания `cargo-tarpaulin` требует аудита переменной среды `$PATH` для обеспечения абсолютной маршрутизации к каталогу `$HOME/.cargo/bin`, а также проверки наличия системных зависимостей уровня компиляции (таких как `libssl-dev` / `OpenSSL`). При использовании CI-систем (например, GitHub Actions) недопустимо использование символа тильды (`~`) для конкатенации путей.
2. В средах `no_std` использование стандартного движка `ptrace` ограничено. Рекомендуется компилировать тестируемые модули с помощью `#[cfg(test)]` как

стандартные бинарные файлы хоста, применять флаг `--engine llvm` для корректного трекинга сложных макросов Verus и гарантировать отсутствие агрессивных оптимизаций (`opt-level=0`), которые маскируют реально исполняемые строки кода. Синтез формальных доказательств Verus для параллельных примитивов синхронизации и всеобъемлющего тестового покрытия Tarpaulin для логики времени выполнения гарантирует, что фундамент операционной системы будет как математически обоснованным, так и эмпирически надежным.

Источники

1. [2303.05491] Verus: Verifying Rust Programs using Linear Ghost Types (extended version), <https://arxiv.org/abs/2303.05491>
2. [PDF] Verus: Verifying Rust Programs using Linear Ghost Types | Semantic Scholar, <https://www.semanticscholar.org/paper/Verus%3A-Verifying-Rust-Programs-using-Linear-Ghost-Lattuada-Hance/0275b04e50fbc16ce63b5711efdd3621bd69733d>
3. Verus: Verifying Rust Programs using Linear Ghost Types - CMU, <https://www.andrew.cmu.edu/user/bparno/papers/verus-ghost.pdf>
4. cargo-tarpaulin - crates.io: Rust Package Registry, <https://crates.io/crates/cargo-tarpaulin>
5. Verifying Rust code with ghost state and invariants - Iris, <https://iris-project.org/workshop-2024/slides/hance.pdf>
6. Intro - Verus Transition Systems, https://verus-lang.github.io/verus/state_machines/
7. Spinlocks Considered Harmful - matklad, <https://matklad.github.io/2020/01/02/spinlocks-considered-harmful.html>
8. The ticket spinlock | The Infinite Loop - WordPress.com, <https://geidav.wordpress.com/2016/04/09/the-ticket-spinlock/>
9. Lecture 18: Fair locks and RCU – CS 161 2019, <https://read.seas.harvard.edu/cs161/2019/lectures/lecture18/>
10. Verifying Concurrent Systems Code - SCS TECHNICAL REPORT COLLECTION, <http://reports-archive.adm.cs.cmu.edu/anon/2024/CMU-CS-24-146.pdf>
11. Proving Refinement in a Rust Verifier - ETH Zürich, https://ethz.ch/content/dam/ethz/special-interest/infk/chair-program-method/pm/documents/Education/Theses/Jan_Schaer_MS_Thesis.pdf
12. `cargo-tarpaulin`: code coverage for Rust - C4DT - EPFL, <https://c4dt.epfl.ch/article/cargo-tarpaulin-code-coverage-for-rust/>
13. Why is my installed command not showing in 'cargo --list'? - Stack Overflow, <https://stackoverflow.com/questions/78374309/why-is-my-installed-command-not-showing-in-cargo-list>
14. Cargo : The term 'cargo' is not recognized as the name of a cmdlet, function, script file, or operable program. | by Crypto Wiki | Medium, <https://medium.com/@cryptowikihere/cargo-the-term-cargo-is-not-recognized-as-the-name-of-a-cmdlet-function-script-file-or-37ae9011e581>
15. cargo install cargo-tarpaulin fails as it is unable to find openssl #1267 - GitHub, <https://github.com/xd009642/tarpaulin/discussions/1267>
16. Tarpaulin flagging line as not covered - Stack Overflow, <https://stackoverflow.com/questions/71180913/tarpaulin-flagging-line-as-not-covered>